C/C++ Engaging OpenMind Software

Compiling Source Code and GNU Make

This page covers the basics of building programs from C source code, and automating this process using GNU Make. It is intended for scientists venturing into scientific programming, to help ease the frustrations that typically come up when starting to work in compiled programming languages.

Preparation

A GCC compiler is needed to compile codes. We load a GCC module when you log in, and you can check which GCC module you have loaded with

```
module list gcc
```

If you need a different version of GCC you can load it with:

```
module load gcc/12.2.0
```

Building a single-file program

Let's start with a simple example: building a "hello world" C program with the GCC compiler. The program (*hello.c*) looks like this:

```
#include <stdio.h>
int main()
{
    printf("Hello World\n");
    return (0);
}
```

To build a working executable from this file, run:

```
gcc hello.c -o hello
```

This command creates an executable with a name of *hello*. Running this command prints the familiar message:

```
$ ./hello
Hello World
```

More happened here behind the scene. In fact, this command wraps up 4 steps of the build process: Preprocess, Compile, Assemble, and Link.

Step 1: Preprocess

In this step, `gcc` calls preprocessing program `cpp` to interpret preprocessor directives and modify the source code accordingly.

Some common directives are:

- `#include`: includes contents of the named file, typically a header file, e.g. `#include <stdio.h>`.
- `#define`: macro substitution, e.g. `#define PI 3.14159`.
- `#ifdef ... #end`: conditional compilation, the code block is included only if a certain macro is defined, e.g:

  ```
  #ifdef TEST_CASE
    a=1; b=0; c=0;
  #endif
  ```

We could perform just this step of the build process like so:

```
cpp hello.c hello.i
```

Examining the output file (`vim hello.i`) shows that the long and messy *stdio.h* header has been appended to our simple code.

Step 2: Compile

In this step, the (modified) source code is translated from the C programming language into assembly code.

Assembly code is a low-level programming language with commands that correspond to machine instructions for a particular type of hardware. It is still just plain text, that says you can read

assembly and write it too if you so desire.

To perform just the compilation step of the build process, we would run:

```
gcc -S -c hello.i -o hello.s
```

Examining the output file (`vim hello.s`) shows processor-specific instructions needed to run our program on this specific system. Interestingly, for such a simple program as ours, the assembly code is actually shorter than the preprocesses source code (though not the original source code).

Step 3: Assemble

Assembly code is then translated into object code. This is a binary representation of the actions your computer needs to take to run your program. It is no longer human-readable, but it can be understood by computers.

To perform just this step of the build process, we would run:

```
gcc -c hello.s -o hello.o
```

You can try to view this object file like we did the other intermediate steps (`vim hello.o`), but the result will not be useful . Your text editor is trying to interpret binary machine language commands as ASCII characters, and (mostly) failing. Perhaps the most interesting result of doing so is that there are intelligable bits --- these are the few variables, etc, that actually are ASCII characters.

Also note that object files are not executables, you can't run them until after the next step.

Step 4: Link

In the final step, `gcc` calls the linker program `ld` to combine the object file with any external functions it needs (e.g. library functions or functions from other source files). In our case, this would include `printf` from the C standard library.

To perform just this step of the build process, we would run:

```
gcc hello.o -o hello
```

This produces the executable `hello` finally.

Building a multi-file program

For most projects in the real world, it is convenient to break up the source code into multiple files. Typically, these include a main function in one file, and one or more other files containing functions / subroutines called by `main()`. In addition, a header file is usually used to share custom data types, function prototypes, preprocessor macros, etc.

As an example, we create several source code files in a directory named *multi_string*, which consists of:

- *main.c*: the main driver function, which calls a subroutine and exits
- *WriteMyString.c*: a module containing the subroutine called by main
- *header.h*: one function prototype and one macro definition

Side note: source codes for the *multi_string* program

main.c:

```
#include "header.h"
#include <stdio.h>
char    *AnotherString = "Hello Everyone";
main()
{
  printf("Running...\n");
  WriteMyString(MY_STRING);
  printf("Finished.\n");
}
```

WriteMyString.c:

```
#include <stdio.h>
extern char *AnotherString;
void WriteMyString(char *ThisString)
{
  printf("%s\n", ThisString);
  printf("Global Variable = %s\n", AnotherString);
}
```

header.h:

```
#define MY_STRING "Hello World"
void WriteMyString();
```

The easiest way to compile such a program is to include all the required source files at the `gcc` command line:

```
gcc main.c WriteMyString.c -o my_string
./my_string
```

It is also quite common to separate out the process into two steps:

1. source code -> object code

```
gcc -c WriteMyString.c
gcc -c main.c
```

2. object code -> executable (or library)

```
gcc WriteMyString.o main.o -o my_string
```

The reason is that this allows you to reduce compiling time by only recompiling objects that need to be updated. This seems silly for a program with only a few source files, but becomes important when many source files are involved. We will use this approach later when we discuss automating the build process.

Including header files

In the above process, it is not necessary to include the header file explicitly on the `gcc` command line. This makes sense since we know that the (bundeled) preprocessing step will append any required headers to the source code before it is compiled.

There is one caveat: the preprocessor must be able to find the header files in order to include them. Our example works because the *header.h* file is in the current directory when we run `gcc`. We can break it by moving the header to a new subdirectory, like so:

```
mkdir include
mv header.h include
gcc main.c WriteMyString.c -o my_string
```

The above commands give the output error:

```
main.c:4:10: fatal error: header.h: No such file or directory
    4 | #include "header.h"
      |          ^~~~~~~~~~
compilation terminated.
```

We can fix this by specifically telling gcc where it can find the requisite headers, using the `-I` flag:

```
gcc -I ./include main.c WriteMyString.c -o my_string
```

This is most often needed in the case where you wish to use external libraries installed in non-standard locations. We will explore this case in the next section.

Linking external libraries

A library is a collection of pre-compiled object files that can be linked into your programs via the linker. In simpler terms, they are machine code files that contain functions / subroutines that you can use in your programs.

Shared libraries vs static libraries

A static library has file extension of *.a* (meaning archive file). When your program links a static library, the machine code of external functions used in your program is copied into the executable. At runtime, everything your program needs is wrapped up inside the executable.

A shared library has file extension of *.so* (meaning shared objects). When your program is linked against a shared library, only a small table is created in the executable. At runtime, the exectutable must be able to locate the functions listed in this table. This is done by the operating system - a process known as dynamic linking.

Static libraries certainly seem simpler, but most programs use shared libraries and dynamic linking. There are several reasons why the added complexity is thought to be worth it:

- Makes executable files smaller and saves disk space, because one copy of a library can be shared between multiple programs.
- Most operating systems allow one copy of a shared library in memory to be used by all running programs, saving memory.
- If your libraries are updated, programs using shared libraries automatically take advantage of these updates, programs using static libraries would need to be recompiled.

Because of the advantage of dynamic linking, GCC will prefer a shared library to a static library if both are available (by default). We will only use shared libraries in the following.

Building with libraries in default locations

Many useful fuctions are provided by libraries in the operating system. These are two widely-used examples:

- `printf()` from the *libc.so* shared library
- `sqrt()` from the *libm.so* shared library

In this section, we will introduce how to build a program with shared libraries in the system default locations. Let's start with an example (*roots.c*) that uses the `sqrt()` function from the math library:

```
#include <stdio.h>
#include <math.h>
void main()
{
    int i;

    printf("\t Number \t\t Square Root of Number\n\n");

    for (i=0; i<=360; ++i)
        printf("\t %d \t\t\t %d \n", i, sqrt((double) i));

}
```

Notice the function `sqrt`, which we use, but do not define. The (machine) code for this function is stored in *libm.so*, and the function definition is stored in the header file *math.h*.

To build successfully, we must:

1. Include the header file for the external library;
2. Instruct the linker to link to the external library.

We build the program using the two-step scheme:

```
gcc -c roots.c
gcc roots.o -lm -o roots
```

The first command preprocesses *roots.c*, appending the header files, and then translates it to object code. This step does need to find the header file, but it does not yet require the library.

The second command links all of the object code into the executable. It does not need to find the header file, which has already been compiled into *roots.o*, but it does need to find the library file.

Library files are linked using the `-l` flag. Their names are given excluding the *lib* prefix and exluding the *.so* suffix, which translates *libm.so* into `m` in this case. So we use `-lm` in the command.

Just as we did above, we can combine the two steps into a single command:

```
gcc roots.c -lm -o roots
```

Finally, we can run the programm:

```
./roots
```

Note

Because we are using shared libraries, the linker must be able to find the linked libraries at runtime, otherwise the program will fail. You can check the libraries required by a program, and whether they are being found correctly or not using the `ldd` command. For out *roots* program, we get the following

```
$ ldd roots
linux-vdso.so.1 (0x00007ffd2c962000)
libm.so.6 => /lib64/libm.so.6 (0x00007fceadbef000)
libc.so.6 => /lib64/libc.so.6 (0x00007fcead82a000)
/lib64/ld-linux-x86-64.so.2 (0x00007fceadf71000)
```

This shows that our executable requires a few basic system libraries such as *libc.so* as well as the math library `libm.so` we explicitly included, and that all of these dependencies are found by the linker.

Side note: where does the preprocessor look to find header files?

The preprocessor will search some default paths for included header files. Before we go down the rabbit hole, it is important to note that you do not have to do this for a typical build, but the commands may prove useful when you are trying to work out why something fails to build.

To look for the header, we can run the following command to show the preprocessor search path:

```
cpp -Wp,-v
```

The output show the paths where GCC will search for header files by default.

Side note: where does the linker look to find libraries?

The linker will search some default paths for library files. Again, it is important to note that you do not have to do this for a typical build, but the commands may prove useful when you are trying to work out why something fails to build.

To look for the library, we can run the following command to get a list of all library files the linker is aware of,

```
ldconfig -p
```

or search that list for the math library we need:

```
ldconfig -p | grep libm.so
```

The latter command gives the output:

```
libm.so.6 (libc6,x86-64, OS ABI: Linux 3.2.0) => /lib64/libm.so.6
```

which shows that the math library is available.

We might also want to peek inside a library file (or any object code for that matter) to see what functions and variables are defined within. We can list all the names, then search for the one we care about, like so:

```
nm /lib64/libm.so.6 | grep " sqrt"
```

The output of this command contains the following line, which shows that it does indeed include something called `sqrt`.

```
000000000000f7d0 W sqrt
```

Building with libraries in non-default locations

In many cases, you may need to use external libraries that are not included in the operating system. These libraries can be built by you or other develepers and they are saved in non-default locations. In this section, we will introduce how to build a program with libraries in non-default locations.

Let's switch to a new example code. We create a source code named *use_ctest.c* that reads the following:

```
#include <stdio.h>
#include "ctest.h"

int main(){
    int x;
    int y;
    int z;
    ctest1(&x);
    ctest2(&y);
    z = (x / y);
    printf("%d / %d = %d\n", x, y, z);
    return 0;
}
```

This code calls two functoins `ctest1` and `ctest2`, which are included in a custom library named *ctest*.

Side note: building a library

In the same level of the main code *use_ctest.c*, we create a directory named *ctest_dir* to save all files related to the library *ctest*.

```
mkdir ctest_dir
```

First, create a subdirectory named `src`,

```
cd ctest_dir
mkdir src
cd src
```

and save the following two source code files in there.

ctest1.c:

```
void ctest1(int *i){
  *i=100;
}
```

ctest2.c:

```
$ cat ctest2.c
void ctest2(int *i){
  *i=5;
}
```

Each code does nothing but defines an interger.

Second, create another subdirectory named `include`,

```
mkdir include
```

and save the following header file in there, *ctest.h*:

```
#ifndef CTEST_H
#define CTEST_H
void ctest1(int *);
void ctest2(int *);
#endif
```

This is for the declaration of the funtcions.

Third, use the following commands to build the shared library named `libctest.so`:

```
gcc -Wall -fPIC -c ctest1.c ctest2.c
```

```
gcc -shared -Wl,-soname,libctest.so -o libctest.so ctest1.o ctest2.o
```

Finally, we move the library to a directory named *lib*,

```
cd ..
mkidr lib
mv src/libctest.so lib
```

Assuming that the library *ctest* has been built (as instructed in the above side note), we will build the program *use_ctest* and fix possbile errors in the process.

First, we start with the simplest command:

```
gcc -c use_ctest.c
```

It fails with an error:

```
use_ctest.c:2:10: fatal error: ctest.h: No such file or directory
```

As the error message indicates, the problem here is that an included header file is not found by the preprocessor. We know that we can use the `-I` flag to fix this problem:

```
gcc -I ctest_dir/include -c use_ctest.c
```

then it creates an object file *use_ctest.o*.

The next step is to use the linker to create an executable. As we have known, we need to explicitly add the library with the `-l` flag,

```
gcc use_ctest.o -lctest -use_ctest
```

but in this case we still get an error:

```
/usr/bin/ld: cannot find -lctest
collect2: error: ld returned 1 exit status
```

Just like for the header, we need to explicitly specify the path to the library file using `-L`:

```
gcc -L ctest_dir/lib  use_ctest.o -lctest -o use_ctest
```

An executable *use_ctest* is created successfully!

Howerver, what happens when we try to run our shiny new executable?

```
$ ./use_ctest
./use_ctest: error while loading shared libraries: libctest.so: cannot open
shared object file: No such file or directory
```

Frustrating? No worry. This is a commonly seen error. We can diagnose this problem by checking to see if the dynamic linker is able to gather up all the dependencies at runtime:

```
$ ldd use_ctest
    linux-vdso.so.1 (0x00007fff56d9d000)
    libctest.so => not found
    libc.so.6 => /lib64/libc.so.6 (0x00007f7f46df6000)
    /lib64/ld-linux-x86-64.so.2 (0x00007f7f471bb000)
```

The output clearly shows that it does not. The problem here is that the dynamic linker will only search the system default paths. There are a few solutions.

- Permanently add our custom library to one of the system default paths. This option needs root permissoins, which is not available for HPC users and thus is not recommended here.
- Specify the location of libraries using the `LD_LIBRARY_PATH` environment variable. `LD_LIBRARY_PATH` contains a colon (:) separated list of directories where the dynamic linker should look for shared libraries. The linker will search these directories before the default system paths. You can define the value of `LD_LIBRARY_PATH` like so:

  ```
  export LD_LIBRARY_PATH=./ctest_dir/lib:$LD_LIBRARY_PATH
  ```

  and then run the program:

  ```
  $ ./use_ctest
  100 / 5 = 20
  ```

  It works!

- Hard-code the location of libraries into the executable. Setting (and forgeting to set) `LD_LIBRARY_PATH` all the time can be tiresome. An alternative approach is to burn the location of the shared libraries into the executable as an `RPATH` or `RUNPATH`. This is done by adding some additional flags for the linker, like so:

  ```
  gcc -L ctest_dir/lib use_ctest.o -lctest -Wl,--rpath=ctest_dir/lib -o
  use_ctest_1
  ```

  We can confirm that this works by running the program:

```
$ ./use_ctest_1
100 / 5 = 20
```

or examining the executable to show that it contains the needed library:

```
$ readelf -d use_ctest_1 |grep ctest
 0x0000000000000001 (NEEDED)             Shared library: [libctest.so]
 0x000000000000000f (RPATH)              Library rpath: [ctest_dir/lib]
```

Automating the build process with GNU Make

The manual build process we used above can become quite tedious in real world. There are many ways that we might automate this process. The simplest would be to write a shell script that runs the build commands each time we invoke it. Let's take the simple *hello.c* program as a test case. Here is a bash shell script (named *build.sh*) to automate the building process,

```
#!/bin/bash
gcc -c hello.c
gcc hello.o -o hello
```

Run it like so:

```
chmod +X build.sh
./build.sh
```

This works fine for small projects, but for large multi-file projects, we would have to compile all the source codes every time we change anything in the codes.

The GNU Make utility provides a useful way around this problem. The solution is that we (the programmers) write a special script that defines all the dependencies between source files, edit one or more files in our project, then invoke Make to re-compile only those files that have been changed.

How GNU Make works

GNU Make is a mini-programming language unto itself. To start, we need to create a file named *Makefile* or *makefile* to define a set of tasks to be executed. For the simple *hello* program, a *Makefile* is like this:

```
hello: hello.o
	gcc hello.o -o hello
```

```
hello.o: hello.c
	gcc -c hello.c

clean:
	rm hello hello.o
```

We can see that each section starts with a line specifyting dependency like so: `target: prerequisites`. The command block that follows will be executed to generate the target if any of the prerequisites have been modified.

> **Note**
>
> Every line in the command block should be started a `tab` character instead of multiple `space` characters.

Once a Makefile is ready, the program can be built by executing one single command,

```
make
```

It looks for the *Makefile* in the same directory and build the targets. The first (top) target will be built by default, or you can specify a specific target to be built,

```
make hello
```

When we run `make`, the computer will take the following actions:

1. Find the default target, which is our executable *hello*.
2. Check if the target file *hello* is up-to-date. A target is considered out-of-date if it does not exist or is older than any of the prerequisites. As *hello* does not exist, so it will be built.
3. The prerequisite of *hello* is *hello.o*, which is also a target, so check if it is up-to-date. As *hello.o* does not exist, so it will be built.
4. The prerequisite of *hello.o* is *hello.c*, which is not a target, so there is nothing left to check. The command `gcc -c hello.c` will be run to create *hello.o*.
5. Now *hello.o* is up-to-date, so the next target *hello* will be built by running the command `gcc hello.o -o hello`.

Note that the command under the *clean* target is not executed by `make`, because it is neither the first target nor an prerequisite of any other target. To bring it up, we need to specify the target name:

```
make clean
```

The command under this target will remove the executable and all of the `.o` files. Note that if all targets are up-to-date, make does not do anything.

Makefile for a multi-file program

Now let's switch to the multi-file example program used in previous sections. A `Makefile` is like this:

```
write: main.o WriteMyString.o
	gcc main.o WriteMyString.o -o write

main.o: main.c header.h
	gcc -c main.c

WriteMyString.o: WriteMyString.c
	gcc -c WriteMyString.c

clean:
	rm write *.o
```

For the first build, `make` builds the targets in the following order: *main.o*, *WriteMyString.o*, and *write*. This compiles all source codes and then links all object files to create the executable.

When the program is rebuilt, `make` will only build the targets whose prerequisites have been modified since the last build. This feature makes the building process efficient for a program with many source files. For example, if *WriteMyString.c* is modified, it is recompiled, while *main.c* is not. If *main.c* or *header.h* is modified, only *main.c* is recompiled, while *WriteMyString.c* is not. In either case, the *write* target will be rebuilt, since either *main.o* or *WriteMyString.o* is updated.

We need to run `make clean` and then run `make`, if we want to completely rebuild evetyhing.

Note: silent mode

By default, `make` prints on the screen all the commands that it executes. To suppress the print, add an `@` before the commands, or turn on the silent mode with the option `-s`:

```
make -s
```

Writing a good Makefile

A *Makefile* could be very compilcated in a practical program with many source files. It is important that the text in a *Makefile* should be as simple and clear as possbile. In this section, we will learn more useful features of GNU Make and write a real-world *Makrefile* at the end.

In our previous examples, you may have noticed that there are many duplications of the same file name or command name. It is convinient to use varialbes in this case. Again, take our multi-file program for example. The *Makefile* can be rewitten as follows,

```
CC=gcc
OBJ=main.o WriteMyString.o
EXE=write

$(EXE): $(OBJ)
        $(CC) $(OBJ) -o $(EXE)

main.o: main.c header.h
        $(CC) -c main.c

WriteMyString.o: WriteMyString.c
        $(CC) -c WriteMyString.c

clean:
        rm $(EXE) *.o
```

Here we have defined the varialbes `CC` for the compiler, `OBJ` for object files, and `EXE` for the executable file. If we want to change the compiler or the file names, we only modify the corresponding variable at one place.

If there are many varialbes to be defined, it is better to write the definition of all variables in another file, and then include the file in Makefile:

```
include ./variables
```

The file *variables* reads the following:

```
CC=gcc
OBJ=main.o WriteMyString.o
EXE=write
```

Furthermore, we can upgrade the *Makefile* to a higher automatic level using the so-called automatic variables:

```
$(EXE): $(OBJ)
        $(CC) $^ -o $@

main.o: main.c header.h
```

```
        $(CC) -c $<

WriteMyString.o: WriteMyString.c
        $(CC) -c $<
```

Here we have used these automatic variables:

- `$@` -- the name of the current target
- `$^` -- the names of all the prerequisites
- `$<` -- the name of the first prerequisite

These variables automatically take the names of current target or prerequisites, no matter what values are assigned to them.

We then notice that the *main.o* and *WriteMyString.o* targets are built by the same command. Is there a way to combine these two duplicated commands into one so as to compile all source code files by only one command line? The answer is yes. It can be done with an implicit pattern rule:

```
%.o: %.c
        $(CC) -c $<

main.o: header.h
```

Here the `%` stands for the same thing in the prerequisites as it does in the target. Usually, any object file with a subfix `.o` has a corresponding source file with a subfix `.c` as an implied prerequisite, so we can use the `%` to represent the name for both files. If a target (e.g. *main.o*) needs additional prerequisites (e.g. *header.h*), write an actionless rule with those prerequisites. We can imagine that applying this impilict rule should significantly simpify a Makefile when there are a large number of source files.

Mostly the target name is a file name. But there are exceptions, such as the *clean* target in this example. The `rm` command will not create any file named `clean`. What if there exists a file named clean in this directory? Let's do an experiment.

```
$ touch clean
$ make clean
make: `clean' is up to date.
```

We can see that the clean target does not work properly. Since it has no prerequisite, the target *clean* will be considered up-to-date, and thus nothing will be done. To solve this issue, we can declare the target to be phony by making it a prerequisite of the special target `.PHONY` as follows:

```
.PHONY: clean
```

A phony target is one that is not really the name of a file; rather it is just a name for a recipe to be executed.

In summary, we write a real-world Makefile like so:

```
include ./variables
.PHONY: clean

$(EXE): $(OBJ)
	$(CC) $^ -o $@

%.o: %.c
	$(CC) -c $<

main.o: header.h

clean:
	rm $(EXE) *.o
```

February 11, 2026